

Федеральное государственное автономное образовательное
учреждение
высшего образования «Санкт-Петербургский национальный
исследовательский Университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3
По вычислительной математике
Вариант №2

Выполнил: Гладышев Иван Станиславович
Группа: Р3213

Принимающий работу: Бострикова Дарья Константиновна

Санкт-Петербург 2026

Содержание

1	Цель лабораторной работы	2
2	Порядок выполнения работы	3
3	Рабочие формулы методов	4
3.1	Метод левых прямоугольников	4
3.2	Метод правых прямоугольников	4
3.3	Метод средних прямоугольников	4
3.4	Метод трапеций	4
3.5	Метод Симпсона (парабол)	4
3.6	Формула Ньютона–Котеса	4
3.7	Правило Рунге	5
3.8	Блок-схема алгоритма численного интегрирования	5
3.9	Несобственные интегралы	6
4	Листинг программы	7
4.1	Класс Integrator	7
5	Результаты выполнения программы	12
5.1	Пример 1. Вычисление собственного интеграла	12
5.2	Пример 2. Вычисление сходящегося несобственного интеграла .	13
5.3	Пример 3. Расходящийся несобственный интеграл	14
6	Вычисление заданного интеграла	15
6.1	Точное вычисление	15
6.2	Метод Ньютона–Котеса ($n = 6$)	16
6.3	Метод средних прямоугольников ($n = 10$)	17
6.4	Метод трапеций ($n = 10$)	18
6.5	Метод Симпсона ($n = 10$)	19
7	Выводы	20

1 Цель лабораторной работы

Найти приближенное значение определенного интеграла с требуемой точностью различными численными методами:

- метод прямоугольников (левые, правые, средние);
- метод трапеций;
- метод Симпсона;
- метод Ньютона–Котеса.

Оценить погрешность вычислений, сравнить полученные результаты с точным значением интеграла.

2 Порядок выполнения работы

Ручной счёт:

1. Вычислить интеграл точно по формуле Ньютона–Лейбница.
2. Вычислить интеграл по формуле Ньютона–Котеса при $n = 6$.
3. Вычислить интеграл по формуле средних прямоугольников при $n = 10$.
4. Вычислить интеграл по формуле трапеций при $n = 10$.
5. Вычислить интеграл по формуле Симпсона при $n = 10$.
6. Сравнить полученные результаты с точным значением. Определить относительную погрешность для каждого метода.

Программная реализация:

1. Реализовать методы прямоугольников (левые, правые, средние), трапеций и Симпсона. Каждый метод оформить в виде отдельной функции.
2. Вычисление значений подынтегральной функции оформить в виде отдельной функции.
3. Для оценки погрешности и завершения вычислительного процесса использовать правило Рунге.
4. Предусмотреть ввод исходных данных (выбор функции, пределы интегрирования, точность) с клавиатуры.
5. Вывести на экран полученное значение интеграла и число разбиений, при котором достигнута требуемая точность.

Необязательное задание:

1. Установить сходимость несобственных интегралов второго рода для двух-трёх функций. Если интеграл расходится, выводить сообщение «Интеграл не существует».
2. Если интеграл сходится, реализовать в программе его вычисление численными методами.
3. Рассмотреть три случая расположения точки бесконечного разрыва подынтегральной функции: в левой границе интегрирования, в правой границе интегрирования, а также внутри отрезка интегрирования.

3 Рабочие формулы методов

3.1 Метод левых прямоугольников

$$\int_a^b f(x)dx \approx h \sum_{i=0}^{n-1} f(x_i), \quad h = \frac{b-a}{n}, \quad x_i = a + i \cdot h$$

3.2 Метод правых прямоугольников

$$\int_a^b f(x)dx \approx h \sum_{i=1}^n f(x_i), \quad h = \frac{b-a}{n}, \quad x_i = a + i \cdot h$$

3.3 Метод средних прямоугольников

$$\int_a^b f(x)dx \approx h \sum_{i=1}^n f\left(x_{i-\frac{1}{2}}\right), \quad h = \frac{b-a}{n}, \quad x_{i-\frac{1}{2}} = a + \left(i - \frac{1}{2}\right) \cdot h$$

3.4 Метод трапеций

$$\int_a^b f(x)dx \approx h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(x_i) \right), \quad h = \frac{b-a}{n}, \quad x_i = a + i \cdot h$$

3.5 Метод Симпсона (парабол)

$$\int_a^b f(x)dx \approx \frac{h}{3} \left(f(a) + f(b) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + 2 \sum_{i=1}^{n/2-1} f(x_{2i}) \right),$$
$$h = \frac{b-a}{n}, \quad n - \text{чётное}$$

3.6 Формула Ньютона–Котеса

$$\int_a^b f(x)dx \approx (b-a) \sum_{i=0}^n H_i^{(n)} f(x_i)$$

где $H_i^{(n)}$ — коэффициенты Котеса. Для $n = 6$ коэффициенты равны:

$$H_0 = H_6 = \frac{41}{840}, \quad H_1 = H_5 = \frac{216}{840}, \quad H_2 = H_4 = \frac{27}{840}, \quad H_3 = \frac{272}{840}$$

3.7 Правило Рунге

Правило Рунге используется для оценки погрешности численного интегрирования. Если I_h и $I_{h/2}$ — приближённые значения интеграла, вычисленные с шагами h и $h/2$, а k — порядок точности метода ($k = 2$ для методов средних прямоугольников и трапеций, $k = 4$ для метода Симпсона), то погрешность оценивается по формуле:

$$R \approx \frac{|I_{h/2} - I_h|}{2^k - 1}$$

Вычислительный процесс завершается, когда $R < \varepsilon$, где ε — заданная точность. При этом в качестве приближённого значения интеграла принимается $I_{h/2}$.

3.8 Блок-схема алгоритма численного интегрирования

Ниже представлена блок-схема алгоритма приближённого вычисления определённого интеграла с использованием правила Рунге для оценки погрешности.

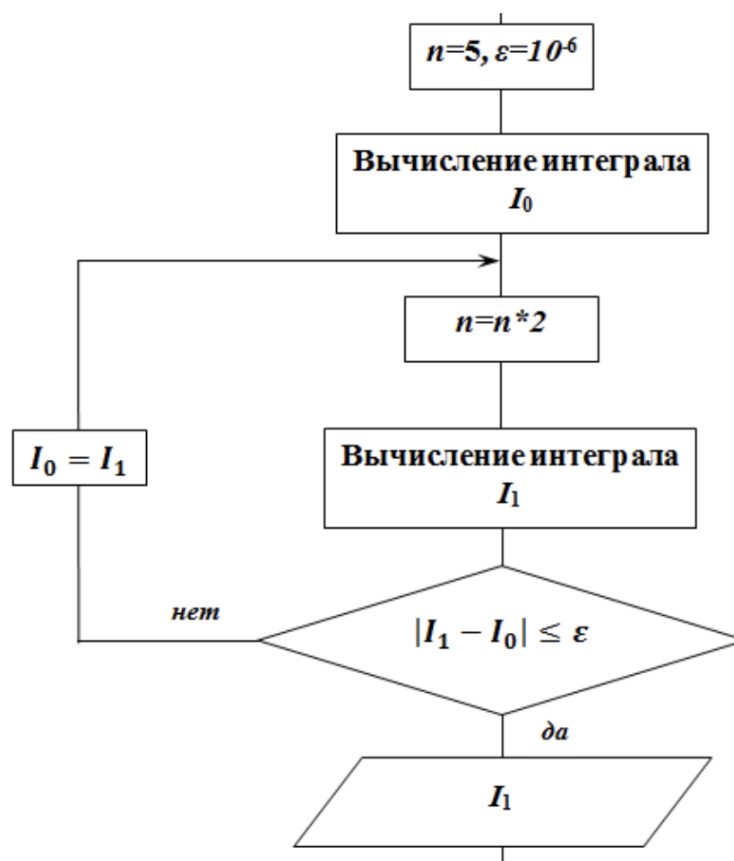


Рис. 1: Блок-схема алгоритма численного интегрирования

3.9 Несобственные интегралы

Разрыв в левой границе (в точке $x = a$):

$$\int_a^b f(x)dx = \lim_{\sigma \rightarrow +0} \int_{a+\sigma}^b f(x)dx, \quad 0 < \sigma < b - a$$

Разрыв в правой границе (в точке $x = b$):

$$\int_a^b f(x)dx = \lim_{\sigma \rightarrow +0} \int_a^{b-\sigma} f(x)dx, \quad 0 < \sigma < b - a$$

Разрыв в обеих границах (в точках $x = a$ и $x = b$):

$$\int_a^b f(x)dx = \lim_{\sigma_1 \rightarrow +0} \lim_{\sigma_2 \rightarrow +0} \int_{a+\sigma_1}^{b-\sigma_2} f(x)dx$$

Разрыв внутри отрезка (в точке $c \in [a, b]$):

$$\int_a^b f(x)dx = \lim_{\sigma_1 \rightarrow +0} \int_a^{c-\sigma_1} f(x)dx + \lim_{\sigma_2 \rightarrow +0} \int_{c+\sigma_2}^b f(x)dx$$

Если указанные пределы существуют и конечны, интеграл называется **сходящимся**. Если хотя бы один из пределов не существует или бесконечен, интеграл называется **расходящимся**.

4 Листинг программы

Программа реализована на языке Java. Основные классы и методы:

4.1 Класс Integrator

Главный класс, содержащий методы численного интегрирования.

```
1 package integration;
2
3 import function.FunctionChoose;
4
5 public final class Integrator {
6     private static final int maxPart = (int) Math.pow(2, 24);
7     private static final int maxDeltaIterations = 60;
8
9     private Integrator() {
10    }
11
12     public static IntegrationResult integrate(
13         FunctionChoose function,
14         double a,
15         double b,
16         IntegrationMethod method,
17         double epsilon,
18         int initialPartitions
19     ) {
20         if (a == b) {
21             return new IntegrationResult(0.0, initialPartitions, 0.0);
22         }
23
24         double left = a;
25         double right = b;
26         if (a > b) {
27             left = b;
28             right = a;
29         }
30
31         FunctionChoose.SingularityCase singularityCase = function.whereGap(
32             left, right);
33         if (singularityCase != FunctionChoose.SingularityCase.NONE) {
34             if (!function.isDesicion(left, right)) {
35                 throw new IllegalArgumentException("
36                     .");
37             }
38             return integrateImproper(function, a, b, method, epsilon,
39                 initialPartitions, singularityCase);
40         }
41
42         return integrateProper(function, a, b, method, epsilon,
43             initialPartitions);
44     }
45
46     private static IntegrationResult integrateProper(
47         FunctionChoose function,
48         double a,
49         double b,
50         IntegrationMethod method,
51         double epsilon,
```

```

48         int initialPartitions
49     ) {
50         int sign = 1;
51         double left = a;
52         double right = b;
53         if (a > b) {
54             sign = -1;
55             left = b;
56             right = a;
57         }
58
59         IntegrationResult result = integrateProperDirect(function, left,
60 right, method, epsilon, initialPartitions);
61         return new IntegrationResult(sign * result.getValue(), result.
62 getParts(), result.getErrorEpsilon());
63     }
64
65     private static IntegrationResult integrateProperDirect(
66         FunctionChoose function,
67         double left,
68         double right,
69         IntegrationMethod method,
70         double epsilon,
71         int initialPartitions
72     ) {
73         int n = initialPartitions;
74         if (method == IntegrationMethod.SIMPSON && n % 2 != 0) {
75             n++;
76         }
77
78         double previous = integrateChoose(function, left, right, n, method);
79
80         while (n <= maxPart / 2) {
81             int newN = n * 2;
82             double current = integrateChoose(function, left, right, newN,
83 method);
84             double rungeError = Math.abs(current - previous) / (Math.pow(2,
85 method.getOrder()) - 1);
86
87             if (rungeError <= epsilon) {
88                 return new IntegrationResult(current, newN, rungeError);
89             }
90
91             n = newN;
92             previous = current;
93         }
94
95         throw new IllegalArgumentException("
96
97             epsilon.");
98     }
99
100     private static IntegrationResult integrateImproper(
101         FunctionChoose function,
102         double a,
103         double b,
104         IntegrationMethod method,
105         double epsilon,
106         int initialPartitions,
107         FunctionChoose.SingularityCase singularityCase

```

```

102     ) {
103         int sign = 1;
104         double left = a;
105         double right = b;
106         if (a > b) {
107             sign = -1;
108             left = b;
109             right = a;
110         }
111
112         double intervalLength = right - left;
113         Double singularPoint = function.getGapPoint();
114         if (singularPoint == null) {
115             throw new IllegalArgumentException("
116
117                 .");
118         }
119
120         double delta = switch (singularityCase) {
121             case AT_A, AT_B -> intervalLength / 4.0;
122             case INSIDE -> Math.min(singularPoint - left, right -
123 singularPoint) / 2.0;
124             case NONE -> throw new IllegalStateException("
125
126                 .");
127         };
128         if (!(delta > 0.0)) {
129             throw new IllegalArgumentException("
130
131                 .");
132         };
133
134         IntegrationResult previous = integrateImproperDelta(
135             function, left, right, singularPoint, singularityCase,
136             method, epsilon / 4.0, initialPartitions, delta
137         );
138
139         for (int i = 0; i < maxDeltaIterations; i++) {
140             delta /= 2.0;
141             IntegrationResult current = integrateImproperDelta(
142                 function, left, right, singularPoint, singularityCase,
143                 method, epsilon / 4.0, initialPartitions, delta
144             );
145
146             double tailError = Math.abs(current.getValue() - previous.
147 getValue());
148             double totalError = tailError + current.getErrorEpsilon();
149             if (totalError <= epsilon) {
150                 return new IntegrationResult(sign * current.getValue(),
151 current.getParts(), totalError);
152             }
153
154             previous = current;
155             if (delta <= Math.ulp(1.0) * Math.max(1.0, intervalLength)) {
156                 break;
157             }
158         }
159
160         throw new IllegalArgumentException("
161
162             .");

```

```

150     }
151
152     private static IntegrationResult integrateImproperDelta(
153         FunctionChoose function,
154         double left,
155         double right,
156         double singularPoint,
157         FunctionChoose.SingularityCase singularityCase,
158         IntegrationMethod method,
159         double epsilon,
160         int initialPartitions,
161         double delta
162     ) {
163         return switch (singularityCase) {
164             case AT_A -> integrateProperDirect(function, left + delta, right
165 , method, epsilon, initialPartitions);
166             case AT_B -> integrateProperDirect(function, left, right - delta
167 , method, epsilon, initialPartitions);
168             case INSIDE -> {
169                 IntegrationResult leftResult = integrateProperDirect(
170 function, left, singularPoint - delta, method, epsilon / 2.0,
171 initialPartitions);
172                 IntegrationResult rightResult = integrateProperDirect(
173 function, singularPoint + delta, right, method, epsilon / 2.0,
174 initialPartitions);
175                 double value = leftResult.getValue() + rightResult.getValue
176 ();
177                 int parts = Math.max(leftResult.getParts(), rightResult.
178 getParts());
179                 double runge = leftResult.getErrorEpsilon() + rightResult.
180 getErrorEpsilon();
181                 yield new IntegrationResult(value, parts, runge);
182             }
183             case NONE -> throw new IllegalStateException("
184 .");
185         };
186     }
187
188     public static double integrateChoose(FunctionChoose function, double a,
189 double b, int n, IntegrationMethod method) {
190         if (n <= 0) {
191             throw new IllegalArgumentException("
192 .");
193         }
194
195         return switch (method) {
196             case LEFT_RECTANGLES -> leftRectangles(function, a, b, n);
197             case RIGHT_RECTANGLES -> rightRectangles(function, a, b, n);
198             case MIDDLE_RECTANGLES -> middleRectangles(function, a, b, n);
199             case TRAPEZOID -> trapezoid(function, a, b, n);
200             case SIMPSON -> simpson(function, a, b, n);
201         };
202     }
203
204     private static double leftRectangles(FunctionChoose function, double a,
205 double b, int n) {
206         double h = (b - a) / n;
207         double sum = 0.0;
208         for (int i = 0; i < n; i++) {
209             sum += function.calculate(a + i * h);
210         }
211     }

```

```

197     }
198     return sum * h;
199 }
200
201 private static double rightRectangles(FunctionChoose function, double a,
double b, int n) {
202     double h = (b - a) / n;
203     double sum = 0.0;
204     for (int i = 1; i <= n; i++) {
205         sum += function.calculate(a + i * h);
206     }
207     return sum * h;
208 }
209
210 private static double middleRectangles(FunctionChoose function, double a
, double b, int n) {
211     double h = (b - a) / n;
212     double sum = 0.0;
213     for (int i = 0; i < n; i++) {
214         sum += function.calculate(a + (i + 0.5) * h);
215     }
216     return sum * h;
217 }
218
219 private static double trapezoid(FunctionChoose function, double a,
double b, int n) {
220     double h = (b - a) / n;
221     double sum = (function.calculate(a) + function.calculate(b)) / 2.0;
222     for (int i = 1; i < n; i++) {
223         sum += function.calculate(a + i * h);
224     }
225     return sum * h;
226 }
227
228 private static double simpson(FunctionChoose function, double a, double
b, int n) {
229     if (n % 2 != 0) {
230         throw new IllegalArgumentException("
                .");
231     }
232
233     double h = (b - a) / n;
234     double sum = function.calculate(a) + function.calculate(b);
235
236     for (int i = 1; i < n; i++) {
237         double coefficient = (i % 2 == 0) ? 2.0 : 4.0;
238         sum += coefficient * function.calculate(a + i * h);
239     }
240
241     return sum * h / 3.0;
242 }
243 }

```

Листинг 1: Класс Integrator (основной)

5 Результаты выполнения программы

Ниже приведены примеры работы программы для различных функций, методов и пределов интегрирования.

5.1 Пример 1. Вычисление собственного интеграла

Численное интегрирование

Выберите функцию:

1. x^2
2. $x^3 - 2x + 1$
3. $\sin(x)$
4. e^x
5. $1 / (x^2 + 1)$
6. $1 / \sqrt{x}$
7. $1 / \sqrt{1 - x}$
8. $1 / \sqrt{|x - 0.5|}$
9. $1 / |x - 0.5|$

Введите номер функции: 1

Выберите метод интегрирования:

1. Метод левых прямоугольников
2. Метод правых прямоугольников
3. Метод средних прямоугольников
4. Метод трапеций
5. Метод Симпсона

Введите номер метода: 3

Введите нижний предел интегрирования a: -1

Введите верхний предел интегрирования b: 1

Введите требуемую точность epsilon: 0.01

Результат:

Функция: x^2

Метод: Метод средних прямоугольников

Интеграл на $[-1.000000; 1.000000] = 0.6640625000$

Число разбиений $n = 16$

Оценка погрешности по правилу Рунге: $2.6041666667e-03$

5.2 Пример 2. Вычисление сходящегося несобственного интеграла

Численное интегрирование

Выберите функцию:

1. x^2
2. $x^3 - 2x + 1$
3. $\sin(x)$
4. e^x
5. $1 / (x^2 + 1)$
6. $1 / \sqrt{x}$
7. $1 / \sqrt{1 - x}$
8. $1 / \sqrt{|x - 0.5|}$
9. $1 / |x - 0.5|$

Введите номер функции: 6

Выберите метод интегрирования:

1. Метод левых прямоугольников
2. Метод правых прямоугольников
3. Метод средних прямоугольников
4. Метод трапеций
5. Метод Симпсона

Введите номер метода: 1

Введите нижний предел интегрирования a: 0

Введите верхний предел интегрирования b: 1

Введите требуемую точность epsilon: 0.01

Результат:

Функция: $1 / \sqrt{x}$

Метод: Метод левых прямоугольников

Интеграл на $[0.000000; 1.000000]$ = 1.9863929464

Число разбиений n = 32768

Оценка погрешности по правилу Рунге: 9.2629790138e-03

5.3 Пример 3. Расходящийся несобственный интеграл

Численное интегрирование

Выберите функцию:

1. x^2
2. $x^3 - 2x + 1$
3. $\sin(x)$
4. e^x
5. $1 / (x^2 + 1)$
6. $1 / \sqrt{x}$
7. $1 / \sqrt{1 - x}$
8. $1 / \sqrt{|x - 0.5|}$
9. $1 / |x - 0.5|$

Введите номер функции: 6

Выберите метод интегрирования:

1. Метод левых прямоугольников
2. Метод правых прямоугольников
3. Метод средних прямоугольников
4. Метод трапеций
5. Метод Симпсона

Введите номер метода: 1

Введите нижний предел интегрирования a: -1

Введите верхний предел интегрирования b: 1

Введите требуемую точность epsilon: 0.01

Ошибка: Не удалось достичь заданной точности. Попробуйте увеличить epsilon.

6 Вычисление заданного интеграла

Задан интеграл:

$$\int_{-3}^{-1} (-3x^3 - 5x^2 + 4x - 2) dx$$

6.1 Точное вычисление

Для точного вычисления определённого интеграла используется формула Ньютона–Лейбница:

$$\int_a^b f(x) dx = F(b) - F(a),$$

где $F(x)$ — первообразная функции $f(x)$.

Находим первообразную:

$$F(x) = \int (-3x^3 - 5x^2 + 4x - 2) dx = -\frac{3x^4}{4} - \frac{5x^3}{3} + 2x^2 - 2x + C$$

Вычисляем значения первообразной в верхнем и нижнем пределах:

$$F(-1) = -\frac{3 \cdot 1}{4} - \frac{5 \cdot (-1)}{3} + 2 \cdot 1 - 2 \cdot (-1) = -\frac{3}{4} + \frac{5}{3} + 2 + 2 = \frac{59}{12}$$

$$F(-3) = -\frac{3 \cdot 81}{4} - \frac{5 \cdot (-27)}{3} + 2 \cdot 9 - 2 \cdot (-3) = -\frac{243}{4} + 45 + 18 + 6 = \frac{33}{4}$$

Точное значение интеграла:

$$I_{\text{точн}} = F(-1) - F(-3) = \frac{59}{12} - \frac{33}{4} = \frac{59}{12} - \frac{99}{12} = -\frac{40}{12} = -\frac{10}{3} \approx -3.3333333333$$

6.2 Метод Ньютона–Котеса (n = 6)

Формула Ньютона–Котеса имеет вид:

$$\int_a^b f(x)dx \approx (b-a) \sum_{i=0}^n H_i^{(n)} f(x_i),$$

где $H_i^{(n)}$ — коэффициенты Котеса. При $n = 6$ коэффициенты равны:

$$H_0 = H_6 = \frac{41}{840}, \quad H_1 = H_5 = \frac{216}{840}, \quad H_2 = H_4 = \frac{27}{840}, \quad H_3 = \frac{272}{840}$$

$$\text{Шаг разбиения: } h = \frac{b-a}{n} = \frac{2}{6} = \frac{1}{3}$$

$$\text{Узлы интегрирования: } x_i = a + i \cdot h = -3 + i \cdot \frac{1}{3}$$

Значения функции в узлах:

i	x_i	$y_i = f(x_i)$
0	-3	22
1	$-\frac{8}{3} \approx -2.666667$	8.6666
2	$-\frac{7}{3} \approx -2.333333$	-0.4444
3	-2	-6
4	$-\frac{5}{3} \approx -1.666667$	-8.6667
5	$-\frac{4}{3} \approx -1.333333$	-9.1111
6	-1	-8

Подставляем в формулу:

$$\begin{aligned} I_{\text{Котес}} &= 2 \cdot \frac{1}{840} (41 \cdot 22 + 216 \cdot 8.6666 + 27 \cdot (-0.4444) + 272 \cdot (-6) \\ &\quad + 27 \cdot (-8.6667) + 216 \cdot (-9.1111) + 41 \cdot (-8)) \end{aligned}$$

После вычислений получаем:

$$I_{\text{Котес}} = -\frac{10}{3} \approx -3.33333333$$

Результат совпадает с точным значением, поэтому относительная погрешность равна 0%.

6.3 Метод средних прямоугольников ($n = 10$)

Формула метода средних прямоугольников:

$$\int_a^b f(x)dx \approx h \sum_{i=1}^n f(x_{i-1/2}),$$

где $h = \frac{b-a}{n}$, $x_{i-1/2} = a + \left(i - \frac{1}{2}\right) h$.

Шаг разбиения: $h = \frac{2}{10} = 0.2$

Вычисляем значения функции в средних точках каждого подынтервала и суммируем их. Полученная сумма:

$$\sum_{i=1}^{10} f(x_{i-1/2}) = -17.1$$

Тогда:

$$I_{\text{ср.пря}} = 0.2 \cdot (-17.1) = -3.42$$

Относительная погрешность:

$$\delta = \frac{|I_{\text{точн}} - I_{\text{ср.пря}}|}{|I_{\text{точн}}|} \cdot 100\% = \frac{0.086667}{3.333333} \cdot 100\% \approx 2.6\%$$

6.4 Метод трапеций (n = 10)

Формула метода трапеций:

$$\int_a^b f(x)dx \approx h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(x_i) \right),$$

где $h = \frac{b-a}{n}$, $x_i = a + i \cdot h$.

Шаг разбиения: $h = 0.2$

Значения функции в узлах:

x_i	y_i
-3.0	22
-2.8	13.456
-2.6	6.528
-2.4	1.072
-2.2	-3.056
-2.0	-6
-1.8	-7.904
-1.6	-8.912
-1.4	-9.168
-1.2	-8.816
-1.0	-8

Сумма значений функции во внутренних узлах:

$$\sum_{i=1}^9 y_i = -22.8$$

Полусумма крайних значений:

$$\frac{y_0 + y_{10}}{2} = \frac{22 + (-8)}{2} = 7$$

Тогда:

$$I_{\text{трап}} = 0.2 \cdot (7 - 22.8) = 0.2 \cdot (-15.8) = -3.16$$

Относительная погрешность:

$$\delta = \frac{|I_{\text{точн}} - I_{\text{трап}}|}{|I_{\text{точн}}|} \cdot 100\% = \frac{0.173333}{3.333333} \cdot 100\% \approx 5.2\%$$

6.5 Метод Симпсона ($n = 10$)

Формула метода Симпсона (при чётном n):

$$\int_a^b f(x)dx \approx \frac{h}{3} \left(f(a) + f(b) + 4 \sum_{\text{нечёт}} f(x_i) + 2 \sum_{\text{чёт}} f(x_i) \right),$$

где $h = \frac{b-a}{n}$, n — чётное.

Шаг разбиения: $h = 0.2$

Используем значения функции из таблицы метода трапеций.

Сумма значений на нечётных индексах ($i = 1, 3, 5, 7, 9$):

$$y_1 + y_3 + y_5 + y_7 + y_9 = 13.456 + 1.072 + (-6) + (-8.912) + (-8.816) = -9.2$$

Сумма значений на чётных индексах ($i = 2, 4, 6, 8$):

$$y_2 + y_4 + y_6 + y_8 = 6.528 + (-3.056) + (-7.904) + (-9.168) = -13.6$$

Подставляем в формулу:

$$I_{\text{Симпс}} = \frac{0.2}{3} (22 + (-8) + 4 \cdot (-9.2) + 2 \cdot (-13.6))$$

$$I_{\text{Симпс}} = \frac{0.2}{3} (14 - 36.8 - 27.2) = \frac{0.2}{3} \cdot (-50) = -\frac{10}{3} \approx -3.33333333$$

Результат совпадает с точным значением, поэтому относительная погрешность равна 0%.

7 Выводы

В ходе выполнения лабораторной работы была реализована программа, которая вычисляет определённые интегралы следующими численными методами: метод левых прямоугольников, метод правых прямоугольников, метод средних прямоугольников, метод трапеций, метод Симпсона. Также программа поддерживает вычисление несобственных интегралов второго рода с автоматической проверкой сходимости и обработкой точек разрыва подынтегральной функции (в левой границе, в правой границе или внутри отрезка интегрирования).

В рамках ручного счёта для интеграла $\int_{-3}^{-1} (-3x^3 - 5x^2 + 4x - 2)dx$ были получены следующие результаты: метод Ньютона–Котеса при $n = 6$ и метод Симпсона при $n = 10$ дали точное значение $-\frac{10}{3}$ (погрешность 0%), метод средних прямоугольников при $n = 10$ дал погрешность около 2.6%, а метод трапеций — около 5.2%. Таким образом, наиболее точными в данном случае оказались метод Симпсона и метод Ньютона–Котеса.

Сравнивая методы между собой, можно отметить, что метод Симпсона обеспечивает высокую точность при относительно небольшом числе разбиений, однако он сложнее в реализации и требует чётного числа разбиений. Методы прямоугольников и трапеций проще в реализации и понимании, но для достижения той же точности им требуется значительно большее число разбиений, что увеличивает время вычислений. Метод Ньютона–Котеса высокого порядка даёт хорошую точность, но требует использования табличных коэффициентов и не всегда удобен для программной реализации. Для практического использования оптимальным является метод Симпсона, сочетающий хорошую точность и приемлемую сложность реализации.